7 참고

전체

# 왜 이 프로그램들인가요? (질문과 답)

영어 원문: WHY-THESE-PROGRAMS.md

## 짧은 버전

*다 읽을 시간이 없나요? 짧은 버전이에요. 전체 안내서는 아래에 있어요.*

- 정원이 왜 지금 모습으로 만들어졌는지 질문과 답으로 설명해요. 모든 답은 예전 서버에서 실제로 일어난 일에서 왔어요.
- Windows가 아니라 Ubuntu인 이유: Windows 업데이트와 멈춘 Docker Desktop 때문에 예전 사이트가 꺼졌어요. 옆에 있던 Linux 서버는 한 번도 멈추지 않았어요.
- Docker는 프로그램마다 밀봉된 상자에 넣어서, 프로그램이 Ubuntu나 서로를 망가뜨릴 수 없어요.
- 포트는 두 개(80과 443)만 열려 있어요. 열린 포트는 하나하나가 인터넷의 누군가가 두드릴 수 있는 문이기 때문이에요.
- 먼저 `.test` 이름으로 연습하세요: Let's Encrypt는 이름마다 일주일에 인증서를 몇 개만 줘요.
- 백업을 많이 하고 노트북 밖에도 사본을 두세요: 같은 노트북의 백업은 실수는 막아 주지만, 노트북이 고장 나면 소용없어요.
- 한 번에 하나만 바꾸고 테스트하세요. 그러면 무엇이 망가뜨렸는지 늘 알 수 있어요.

---

이 페이지는 정원이 왜 지금 모습으로 만들어졌는지 질문과 답으로 설명해요. 모든 답은 정원이 예전 서버에서 돌아가는 동안 실제로 일어난 일에서 왔어요. 어떤 교훈은 배우는 데 몇 시간, 며칠이 걸렸어요. 그 교훈을 그냥 가져가면 돼요.

각 답의 끝에는 **더 읽기**(공식 페이지)가 있어서 직접 확인할 수 있어요.

---

## 컴퓨터

**질문: 왜 Windows가 아니라 Ubuntu인가요?** 답: 예전에는 웹사이트가 Windows PC의 Docker Desktop 안에서 돌아갔어요. 어느 날 밤 Windows 업데이트가 새벽 2시 30분쯤 PC를 저절로 다시 시작해서 사이트가 꺼졌어요. 또 한 번은 Docker Desktop의 엔진이 멈췄어요: 모든 요청이 오류가 났는데, 로그는 하나도 남기지 않았어요. 공개 사이트 두 개가 함께 꺼졌어요. 그 옆의 Linux 서버는 한 번도 멈추지 않았어요. 그래서 정원은 Linux에서 돌아가요. Ubuntu **LTS**("long-term support", 장기 지원)는 5년 동안 무료 보안 업데이트를 받고, 인터넷의 안내서 대부분이 Ubuntu 기준으로 쓰여 있어요. 더 읽기: https://ubuntu.com/about/release-cycle

**질문: 왜 데스크톱이 아니라 오래된 노트북인가요?** 답: 서버는 항상 켜져 있어야 해요. 노트북은 전기를 적게 쓰고, 배터리가 작은 예비 전원 역할을 해요. 설치 프로그램은 뚜껑을 닫아도 노트북이 깨어 있게 해요. 데스크톱, Raspberry Pi, 다른 컴퓨터를 쓰고 싶다면 메뉴에서 "another computer"(다른 컴퓨터)를 고르고 그 주소를 입력하세요. 더 읽기: https://releases.ubuntu.com/24.04/ (시험한 버전. 이유는 안내서 01)

**질문: 왜 노트북에 고정(예약된) 주소가 필요한가요?** 답: 공유기(router)는 인터넷 방문자를 주소 하나로 전달해요. 노트북이 다시 시작한 뒤 새 주소를 받으면, 방문자는 아무 데도 못 가거나 엉뚱한 컴퓨터로 가요. 공유기에서 주소를 예약하면 그런 일이 없어요. 주소가 바뀌면 점검표가 알려줘요. 더 읽기: `START-HERE/14-reserve-your-ip.md` (한국어: START-HERE/ko/14-reserve-your-ip.md)

## Docker와 프로그램

**질문: Docker가 무엇이고, 왜 쓰나요?** 답: Docker는 각 프로그램을 자기만의 밀봉된 상자에서 실행해요. 이 상자를 컨테이너라고 불러요. 상자 안에는 프로그램에 필요한 것이 모두 들어 있어요. 프로그램이 Ubuntu나 서로를 망가뜨릴 수 없고, 파일 하나(`docker-compose.yml`)가 모두를 어떻게 시작할지 정해요. 예전 서버도 이런 식으로 많은 컨테이너를 돌렸어요. 거기서 배운 것은 쓰는 것만 남기라는 거예요: 한때 컨테이너가 백 개가 넘었고, 그중 절반이 무슨 일을 하는지 아무도 몰랐어요. 내 정원에는 몇 개만 있고, 하나하나 다 설명되어 있어요. 더 읽기: https://docs.docker.com/get-started/docker-overview/

**질문: 왜 MariaDB인가요?** 답: 정원은 MySQL로 만들어졌어요. MariaDB는 MySQL을 처음 만든 사람들이 만든 무료 커뮤니티 버전이에요. 똑같은 명령을 그대로 알아들어요. 더 읽기: https://mariadb.org/about/

**질문: 왜 웹사이트를 Rust로 만들었나요?** 답: 처음에는 PHP였어요. 한 페이지씩 Rust로 옮겼고, 다음 페이지를 시작하기 전에 매번 예전 페이지와 비교해서 확인했어요. Rust는 메모리를 적게 써요.

오래된 노트북에서는 이것이 중요해요. 또 프로그램을 만들기도 전에 많은 것을 검사해요. 예를 들어 페이지가 없는 항목을 쓰면, 방문자 앞에서 페이지가 고장 나는 대신 빌드가 멈추고 알려 줘요. 겉모습, 주소(QR 코드에 인쇄된 `.php` 주소까지), 휴대폰 앱은 모두 그대로예요. 더 읽기: https://www.rust-lang.org/learn, `docs/rust-port/CODE-TOUR.md`

## 정문(front door)

**질문: 왜 Caddy인가요?** 답: Caddy는 HTTPS 자물쇠 인증서를 스스로 받아 오고 스스로 갱신해요. 설정 파일이 짧고 읽기 쉬워요. Apache와 nginx도 같은 일을 할 수 있지만 단계가 더 많아요. (`lab/06`은 배우기 위해 그 옆에 Apache를 설치해요.) 내 Caddy는 Docker가 아니라 Ubuntu에서 바로 돌아가요. 그래서 예전 서버의 함정을 피해요: 컨테이너 안의 Caddy는 설정 파일을 바꾼 뒤에도 예전 사본을 계속 읽었어요. 더 읽기: https://caddyserver.com/docs/

**질문: 왜 포트를 두 개만 여나요?** 답: 열린 포트는 모두 인터넷의 누군가가 두드릴 수 있는 문이에요. 포트 80과 443은 Caddy로 가고, Caddy가 각 방문이 어디까지 갈 수 있는지 정해요. 나머지(웹사이트, 데이터베이스, 위키, 도구)는 모두 노트북 자신에서만 들어요. 예전 서버에도 같은 이유로 "새 포트를 열지 않는다"는 규칙이 있었어요. 더 읽기: https://ubuntu.com/server/docs/how-to/security/firewalls/

**질문: 왜 Caddy가 방문자가 보낸 X-Auth-Email 같은 헤더를 지우나요?** 답: 정원 도구는 "이 사람이 누구인지" 알려 주는 헤더를 믿어요. 하지만 로그인 페이지가 넣은 헤더만 믿어야 해요. 방문자가 그 헤더를 직접 보내서 나인 척할 수도 있어요. Caddy는 방문을 넘기기 전에 그런 헤더를 모두 버려요. 이것은 예전 서버의 설정에서 가져왔어요. 더 읽기: https://caddyserver.com/docs/caddyfile/directives/request_header

**질문: 왜 로그인 페이지(Authelia)가 따로 있나요?** 답: 정원 도구에는 자체 로그인이 없어요. 예전 서버에서는 그 앞에 로그인 관문이 있었고, Authelia가 같은 일을 해 줘요. 설명서가 잘 되어 있는 유명한 오픈 소스 프로그램이라서 언제든지 찾아볼 수 있어요. 비밀번호 추측도 느리게 만들어요: 2분 안에 5번 틀리면 그 이름이 10분 동안 잠겨요. 웹사이트는 식물을 편집하기 위한 자체 로그인을 그대로 가지고 있어요. 더 읽기: https://www.authelia.com/overview/prologue/introduction/

**질문: 왜 No-IP이고, 왜 매달 이름을 확인해야 하나요?** 답: 집의 인터넷 주소는 바뀔 수 있어요. No-IP 업데이트 프로그램은 몇 분 안에 새 주소를 No-IP에 알려 줘요. `junsgarden.zapto.org`는 무료 No-IP 이름이고, 무료 이름은 30일마다 확인해야 해요(No-IP가 이메일로 링크를 보내 줘요). 확인을 건너뛰고 싶다면, 유료 No-IP 요금제나 내 도메인(1년에 약 12달러)이 있으면 돼요. 더 읽기: https://www.noip.com/support/knowledgebase/

**질문: 왜 먼저 ".test" 이름으로 연습하나요?** 답: 무료 인증서 서비스(Let's Encrypt)는 한 이름에 일주일 동안 인증서를 몇 개만 줘요. 같은 실수를 몇 번 반복하면 며칠 동안 막힐 수 있어요. `.test` 이름은 Caddy가 직접 만든 연습용 인증서를 써서, 실수를 마음껏 해도 괜찮아요. 더 읽기: https://letsencrypt.org/docs/rate-limits/

## 안전하게 지키기

**질문: 왜 모든 비밀번호가 열어 볼 필요도 없는 `.env` 파일 하나에 있나요?** 답: 코드 안의 비밀번호는 실수로 공유되기 쉬워요. 예전 서버에서 비밀번호가 실수로 화면에 나와서 모든 곳에서 바꿔야 했던 적이 있어요. 그래서 비밀번호는 `.env`에만 있고, 그 파일은 본인만 읽을 수 있고, 설치 프로그램은 비밀번호를 한 번도 보여 주지 않고 만들어요. 설정(메뉴 `8`)이 대신 바꿔 줘요. `.env`를 누군가에게 보여 준 적이 있다면, 설정의 "Renew the internal database passwords"(내부 데이터베이스 비밀번호 새로 만들기)를 쓰세요. 더 읽기: https://owasp.org/www-project-top-ten/

**질문: 왜 백업이 이렇게 많고, 왜 노트북 밖으로 복사하나요?** 답: 예전에는 사진 860 GB가 사본 하나뿐이었는데, 같은 시기에 저장 장치의 디스크 하나가 고장 나고 있었어요. 사람들은 이렇게 모든 것을 잃어요. 같은 노트북 안의 백업은 실수는 막아 주지만, 노트북이 고장 나는 것은 막지 못해요. 그래서: 매일 밤 백업, 그리고 가끔 USB 메모리나 다른 컴퓨터에 사본. 더 읽기: https://www.backblaze.com/blog/the-3-2-1-backup-strategy/

**질문: 왜 모든 백업에 MANIFEST와 SHA256SUMS가 있나요?** 답: MANIFEST(목록)는 안에 무엇이 있는지 알려 줘요. SHA256 지문(fingerprint)은 빠지거나 망가진 것이 없다는 것을 증명해요. 복원은 아무것도 건드리기 전에 이것을 먼저 확인해요. 더 읽기: https://www.gnu.org/software/coreutils/manual/html_node/sha2-utilities.html

**질문: 왜 무료 AI만 쓰고 유료 키는 안 쓰나요?** 답: 유료 키는 무언가가 부를 때마다 돈이 들고, 실수 하나로 큰돈이 나갈 수 있어요. 정원은 무료 등급을 써요: Google Gemini 무료 키와 OpenRouter의 무료 모델. 무료 서비스는 가끔 바빠서, 정원 도구는 실패하는 대신 작업을 대기열에 넣고 나중에 다시 시도해요. 집에 있는 성능 좋은 컴퓨터의 로컬 AI가 비공개 선택지예요(안내서 15). 더 읽기: https://ai.google.dev/gemini-api/docs/rate-limits

## 작업하기

**질문: 왜 "한 번에 하나만 바꾸고, 그다음 테스트"인가요?** 답: 한번은 주석 하나를 잘못된 곳(데이터베이스 명령 안)에 넣었더니, 모든 로그인이 조용히 고장 났어요. 테스트만이 그것을 잡아냈어요. Rust로 옮기는 일도 한 페이지씩, 매번 테스트 사이트와 되돌아가는 방법을 두고 했어요. 하나

만 바꾸고 테스트하면, 무엇이 고장 냈는지 항상 알 수 있어요. 더 읽기: `START-HERE/11-maintain-your-site.md`

**질문: 왜 git인가요?** 답: git은 저장한 모든 버전을 보관해요. `git diff`는 무엇을 바꿨는지 정확히 보여 주고, `git checkout -- FILE`은 그것을 되돌려요. 모든 편집 아래에 있는 안전망이에요. 더 읽기: https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F

**질문: 왜 Windows가 아니라 Ubuntu에서 파일을 고치나요?** 답: Windows와 Linux는 텍스트 파일의 줄 끝을 다르게 표시해요. 예전에 Windows의 git이 저장할 때 몇몇 파일을 조용히 바꿨고, PDF 몇 개까지 망가뜨렸어요. Windows 줄 끝을 가진 셸 스크립트는 Linux에서 실패해요. 노트북에서 고치거나, git이 줄 끝을 건드리지 않도록 설정하세요. 더 읽기: https://docs.github.com/en/get-started/getting-started-with-git/configuring-git-to-handle-line-endings

**질문: 왜 한 번에 목표 하나인가요?** 답: 예전 서버에서는 신나는 추가 기능에 며칠을 쓰고 결국 버린 적이 있어요. 작동하는 것 하나를 끝내는 것이 다섯 개를 시작하는 것보다 나아요. 새 아이디어는 "나중에" 목록에 적어 두세요.

**질문: 막히면 어떻게 하나요?** 답: 메뉴 `H`가 도움 파일을 만들어요: 무슨 일이 있었고 설치가 어디서 멈췄는지, 비밀번호는 빼고요. 먼저 읽어 본 다음, Gemini에 붙여 넣거나 믿을 수 있는 사람에게 보여 주세요. 점검표(메뉴 `5`)가 무엇부터 고쳐야 하는지 알려 줘요.